Fast and scalable general geospatial regridding

Justus Magin[1,2]
[1]LOPS, Brest, France [2]xarray project

Regridding

- Interpolation from one grid to another
- Important operation when working with gridded data on different grids
- Differences: grid type, orientation, resolution
- Point-based and Area-based interpolation algorithms

Offline Regridding

Weights only depend on the geometry.

Fundamental steps:

1. Search cells to aggregate
2. Compute weights
3. Apply weights as a sparse matrix multiplication

Missing features in the ecosystem

- No native support for larger-than-memory grids
- Specialized to certain types of grids (rectilinear / curvilinear)
- Installation is hard, even with conda

Design Principles

- Represent cells as cell boundary polygons
- Implementation on top of the georust[1] ecosystem
- Use an RTree[2] to quickly search cells to aggregate
- Search index in a separate package for reuse (`grid-indexing`)
- `xarray`[3] interface that allows choosing algorithms per variable
- `dask`[4]-based distributed workflow for larger-than-memory grids

Future improvements

- Spherical / ellipsoidal geometry
- Optimization of the dask-based implementation
- Try to reduce the memory footprint using meshes
- Implementation of missing parts in the ecosystem:
  - Serializing / deserializing (chunked) polygon arrays to disk
  - Serializing / deserializing sparse arrays to disk
  - Multi-dimensional geoarrow

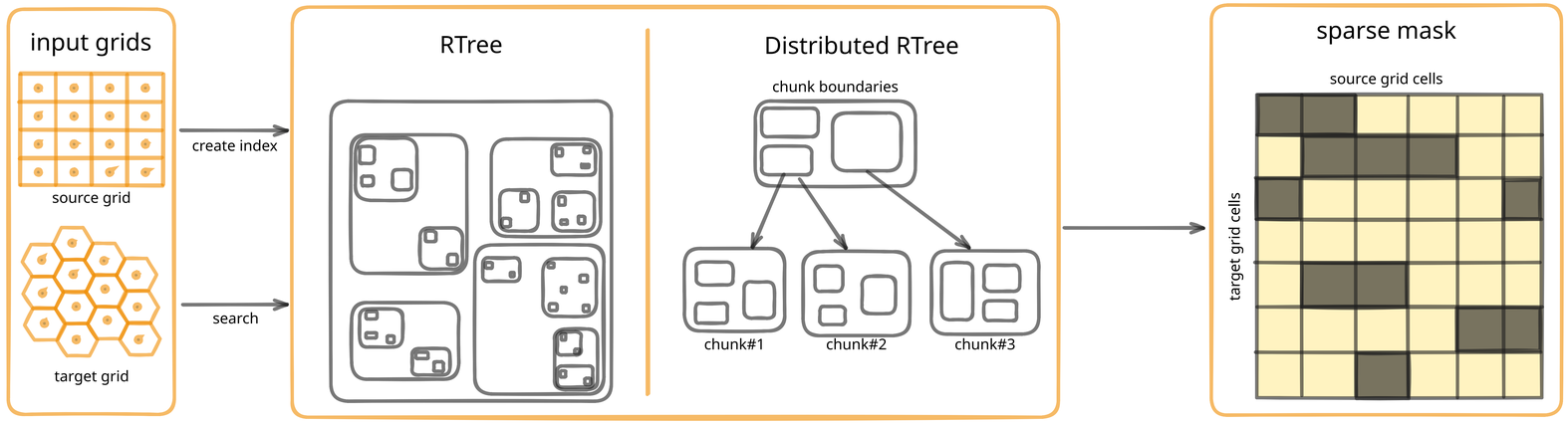
References

[1]https://georust.org
[2]https://crates.io/crates/rstar
[3]https://xarray.dev
[4]https://www.dask.org

`grid-weights` is a new regridding library for python that supports arbitrary grid geometries and grid sizes

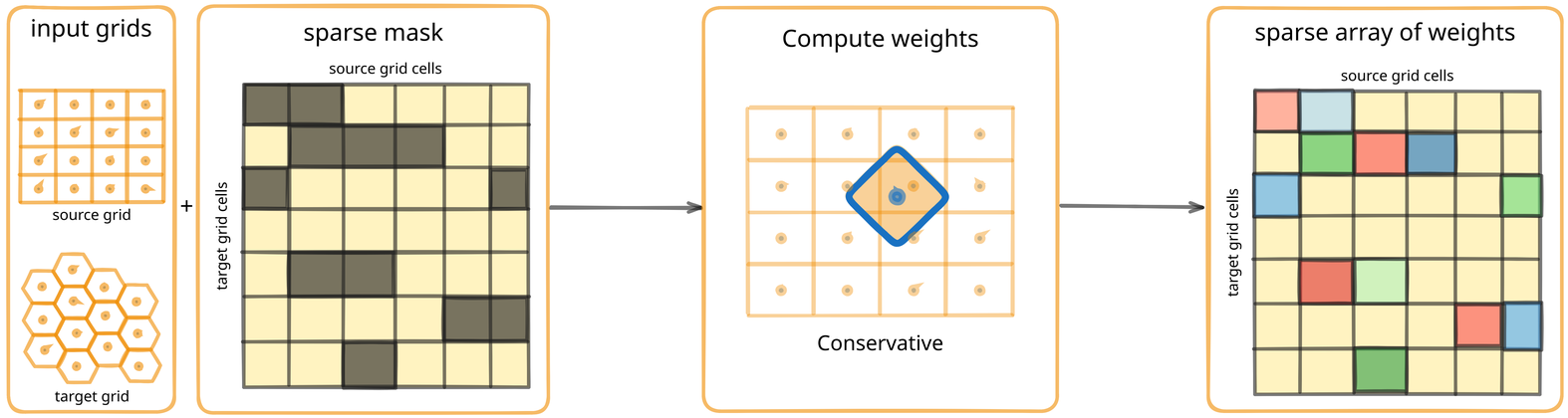
Search

(grid-indexing)



Weights

(grid-weights)



API

```python
algorithms = grid_weights.Algorithms.by_variable(ds, default="conservative")
indexed_cells = grid_weights.create_index(source_geoms).query(
    target_geoms, methods=algorithms.indexing_methods()
)
weights = grid_weights.weights(source_geoms, target_geoms, indexed_cells)

regridded = algorithms.regrid(ds, weights)
```